

BỘ NÔNG NGHIỆP VÀ PHÁT TRIỂN NÔNG THÔN
PHÂN HIỆU TRƯỜNG ĐẠI HỌC THỦY LỢI



ĐỒ ÁN MÔN HỌC MÁY

Tên đề tài: Healthy vs. Stressed Lettuce Image Dataset

Sinh viên thực hiện	
2151067060	Võ Hoàn Lạc
2351067109	Phùng Trí Quân
2351067121	Nguyễn Anh Vũ
Giảng viên hướng dẫn	Ths. Vũ Thị Hạnh

HỒ CHÍ MINH, NĂM 2025

LỜI CAM ĐOAN

Tác giả xin cam đoan đây là báo cáo môn học của bản thân tác giả. Các kết quả trong báo cáo môn học này là trung thực, và không sao chép từ bất kỳ một nguồn nào và dưới bất kỳ hình thức nào. Việc tham khảo các nguồn tài liệu (nếu có) đã được thực hiện trích dẫn và ghi nguồn tài liệu tham khảo đúng quy định.

LỜI CẢM ƠN

Tác giả xin trân trọng cảm ơn giảng viên Vũ Thị Hạnh đã đóng góp ý kiến cho việc soạn thảo tài liệu Hướng dẫn Trình bày Đồ án môn học này.

Nhóm sinh viên thực hiện

Võ Hoàn Lạc

Phùng Trí Quân

Nguyễn Anh Vũ

Lời mở đầu:

Tôi xin được mở đầu bài viết này bằng sự tôn trọng và tinh thần nghiêm túc trong việc trình bày đề tài nghiên cứu của mình đến thầy/cô. Đề tài “Phân loại rau xà lách khỏe mạnh và bị stress dựa trên bộ dữ liệu ảnh (Healthy vs. Stressed Lettuce Image Dataset)” không chỉ mang ý nghĩa học thuật mà còn có giá trị

thực tiễn cao trong lĩnh vực nông nghiệp thông minh và ứng dụng trí tuệ nhân tạo.

Khi lựa chọn đề tài này, tôi nhận thấy đây là một cơ hội để tiếp cận sâu hơn việc áp dụng thị giác máy tính và học máy vào bài toán thực tế, cụ thể là đánh giá tình trạng sinh trưởng của cây trồng thông qua hình ảnh. Việc phân biệt giữa xà lách khỏe mạnh và xà lách bị stress không chỉ giúp nâng cao năng suất canh tác mà còn hỗ trợ người nông dân trong việc phát hiện sớm các vấn đề về môi trường, dinh dưỡng hoặc sâu bệnh.

Trong quá trình nghiên cứu, tôi không chỉ tập trung vào việc xây dựng mô hình phân loại ảnh mà còn chú trọng đến việc hiểu rõ dữ liệu, tiền xử lý hình ảnh và phân tích những đặc trưng ẩn phản ánh tình trạng sinh lý của cây xà lách. Đối với một sinh viên, đề tài này không đơn thuần là áp dụng thuật toán, mà còn là quá trình đặt ra những câu hỏi mang tính cốt lõi như: “Những đặc điểm nào trong hình ảnh thể hiện sự stress của cây?” và “Làm thế nào để mô hình có thể hỗ trợ hiệu quả cho sản xuất nông nghiệp?”

Thông qua đề tài, tôi đã nhận thức rõ hơn vai trò của trí tuệ nhân tạo trong việc hướng tới một nền nông nghiệp bền vững, chính xác và tiết kiệm nguồn lực. Sự kết hợp giữa kiến thức lý thuyết và dữ liệu thực tế đã mang lại nhiều thách thức nhưng đồng thời cũng tạo nên sự hứng thú và động lực trong suốt quá trình thực hiện nghiên cứu.

Cuối cùng, tôi rất mong nhận được sự hướng dẫn và những ý kiến đóng góp quý báu từ thầy/cô để bài nghiên cứu của tôi được hoàn thiện hơn cả về nội dung lẫn phương pháp tiếp cận.

MỤC LỤC

LỜI CAM ĐOAN.....	2
LỜI CẢM ƠN.....	3
Lời mở đầu:.....	4
PHẦN 1: TỔNG QUAN VỀ ĐỀ TÀI.....	6
1.1 Lý do chọn đề tài:.....	6

1.2	Giới thiệu về bộ dữ liệu:	6
1.3	Mục tiêu và phạm vi của đề tài:	7
Phần 2: CỞ SỞ LÝ THUYẾT		7
2.1	Khai phá dữ liệu	7
2.1.1	Học máy	7
2.1.2	Quy trình học máy trong CSDL	8
2.1.3	Các chức năng cụ thể của học máy	9
2.1.4	Các ứng dụng của học máy	9
2.2	Data Preprocessing(Tiền xử lý dữ liệu)	10
2.3	Thuật toán	14
2.3.1	Decision Tree	14
2.3.2	Naive Bayes	18
2.3.3	Random Forest	20
PHẦN 3: TRIỂN KHAI ĐỀ TÀI		21
3.1	Chuẩn bị dữ liệu	21
3.2	Kiểm tra và xử lý dữ liệu	23

PHẦN 1: TỔNG QUAN VỀ ĐỀ TÀI

1.1 Lý do chọn đề tài:

Sau một thời gian tham khảo các đề tài liên quan đến bài toán dự đoán và nhận dạng trên Kaggle cũng như các nền tảng học máy khác, chúng em đã quyết định lựa chọn đề tài “Phân loại xà lách khỏe mạnh và xà lách bị stress dựa trên bộ dữ liệu ảnh (Healthy vs. Stressed Lettuce Image Dataset)” vì những lý do sau:

Thứ nhất, bộ dữ liệu Healthy vs. Stressed Lettuce Image Dataset cung cấp tập hợp hình ảnh phong phú về tình trạng sinh trưởng của cây xà lách trong các điều kiện khác nhau. Điều này tạo điều kiện cho chúng em thực hiện phân tích và xây dựng mô hình phân loại

dựa trên đặc trưng hình ảnh, từ đó nhận diện sự khác biệt giữa cây khỏe mạnh và cây bị stress.

Thứ hai, bài toán phân loại tình trạng cây trồng là một vấn đề mang tính thực tiễn cao và có nhiều ứng dụng trong lĩnh vực nông nghiệp thông minh. Việc phát hiện sớm tình trạng stress của cây giúp người trồng có thể kịp thời điều chỉnh chế độ chăm sóc, tưới tiêu và dinh dưỡng, góp phần nâng cao năng suất và chất lượng nông sản.

Thứ ba, việc làm việc với dữ liệu ảnh thực tế giúp sinh viên chúng em trau dồi và phát triển các kỹ năng quan trọng như tiền xử lý hình ảnh, trích xuất đặc trưng và xây dựng mô hình học máy. Đồng thời, đề tài tạo cơ hội để chúng em tiếp cận và áp dụng các thuật toán Machine Learning và Deep Learning, đặc biệt là các mô hình mạng nơ-ron tích chập (CNN) trong bài toán phân loại ảnh.

Cuối cùng, do phần lớn các nhóm khác lựa chọn các đề tài liên quan đến nhận dạng đối tượng hoặc phân loại thông thường, chúng em quyết định chọn một đề tài mang tính ứng dụng nông nghiệp nhằm tạo sự đa dạng cho các hướng tiếp cận trong báo cáo môn học. Chúng em hy vọng rằng kết quả của đề tài không chỉ là thành quả học tập của nhóm mà còn mang lại giá trị tham khảo và ý nghĩa thực tiễn cho cộng đồng cũng như lĩnh vực nông nghiệp mà chúng em hướng đến. Đây cũng chính là nguồn động viên lớn giúp chúng em không ngừng nỗ lực để đạt được những kết quả tích cực và bổ ích nhất từ đề tài này.

1.2 Giới thiệu về bộ dữ liệu:

Bộ dữ liệu Healthy vs. Stressed Lettuce Image Dataset là một tập hợp dữ liệu ảnh, được xây dựng nhằm phục vụ cho bài toán phân loại tình trạng sinh trưởng của cây xà lách. Dữ liệu được chia thành hai lớp chính, đại diện cho trạng thái sức khỏe của cây trồng.

Thông tin dữ liệu :

- Nguồn dữ liệu: Kaggle
- Link đến Dataset:

<https://www.kaggle.com/datasets/ashimstha/lettuce-health-compiled-dataset>

1.3 Mục tiêu và phạm vi của đề tài:

Mục tiêu của đề tài là xây dựng một mô hình phân loại hình ảnh nhằm nhận diện tình trạng cây xà lách, phân biệt giữa xà lách khỏe mạnh và xà lách bị stress dựa trên bộ dữ liệu ảnh Healthy vs. Stressed Lettuce Image Dataset. Mô hình hướng tới việc hỗ trợ giám sát sức khỏe cây trồng một cách tự động thông qua hình ảnh.

Đề tài tập trung vào các bước chính gồm tiền xử lý dữ liệu ảnh, xây dựng và huấn luyện mô hình học máy/học sâu, thực hiện dự đoán trên tập dữ liệu kiểm tra và đánh giá mô hình bằng các chỉ số cơ bản như độ chính xác và ma trận nhầm lẫn.

Phần 2: CỞ SỞ LÝ THUYẾT

2.1 Khai phá dữ liệu

2.1.1 Học máy

- Tổng quan Học máy (Machine Learning) là một ngành khoa học nghiên cứu các thuật toán cho phép máy tính có thể học được các khái niệm (concept).

- Phân loại: Có hai loại phương pháp học máy chính

+ Phương pháp quy nạp: Máy học/phân biệt các khái niệm dựa trên dữ liệu đã thu thập được trước đó. Phương pháp này cho phép tận dụng được nguồn dữ liệu rất nhiều và sẵn có.

+ Phương pháp suy diễn: Máy học/phân biệt các khái niệm dựa vào các luật. Phương pháp này cho phép tận dụng được các kiến thức chuyên ngành để hỗ trợ máy tính.

- Hiện nay, các thuật toán đều cố gắng tận dụng được ưu điểm của hai phương pháp này.

- Các ngành khoa học liên quan:

+ Lý thuyết thống kê: các kết quả trong xác suất thống kê là tiền đề cho rất nhiều phương pháp học máy. Đặc biệt, lý thuyết thống kê cho phép ước lượng sai số của các phương pháp học máy.

+ Các phương pháp tính: các thuật toán học máy thường sử dụng các tính toán số thực/số nguyên trên dữ liệu rất lớn. Trong đó, các bài toán như: tối ưu có/không ràng buộc, giải phương trình tuyến tính v.v được sử dụng rất phổ biến.

+ Khoa học máy tính: là cơ sở để thiết kế các thuật toán, đồng thời đánh giá thời gian chạy, bộ nhớ của các thuật toán học máy.

Các nhóm giải thuật học máy:

+ Học có giám sát: Máy tính được xem một số mẫu gồm đầu vào (input) và đầu ra (output) tương ứng trước. Sau khi học xong các mẫu này, máy tính quan sát một đầu vào mới và cho ra kết quả.

+ Học không giám sát: Máy tính chỉ được xem các mẫu không có đầu ra, sau đó máy tính phải tự tìm cách phân loại các mẫu này và các mẫu mới.

+ Học nửa giám sát: Một dạng lai giữa hai nhóm giải thuật trên.

+ Học tăng cường: Máy tính đưa ra quyết định hành động (action) và nhận kết quả phản hồi (response/reward) từ môi trường (environment). Sau đó máy tính tìm cách chỉnh sửa cách ra quyết định hành động của mình.

2.1.2 Quy trình học máy trong CSDL

1. Thu Thập Dữ Liệu: Thu thập dữ liệu liên quan đến vấn đề cụ thể mà bạn muốn giải quyết. Dữ liệu này có thể bao gồm thông tin từ CSDL hoặc từ các nguồn khác nhau như API, bảng tính, v.v.

2. Tiền Xử Lý Dữ Liệu: Làm sạch và chuẩn hóa dữ liệu để đảm bảo rằng nó phù hợp với mô hình học máy. Bao gồm xử lý giá trị thiếu, chuyển đổi kiểu dữ liệu, và thực hiện các biện pháp tiền xử lý khác.

3. Chia Dữ Liệu: Phân chia dữ liệu thành tập huấn luyện, tập kiểm thử và tập validation. Tập huấn luyện được sử dụng để huấn luyện mô hình, tập kiểm thử để đánh giá hiệu suất, và tập validation để điều chỉnh các siêu tham số.

4. Chọn Mô Hình: Chọn một mô hình học máy phù hợp với vấn đề của bạn. Các loại mô hình phổ biến bao gồm cây quyết định, máy vector hỗ trợ, mạng nơ-ron, và nhiều loại mô hình khác.

5. Huấn Luyện Mô Hình: Sử dụng tập huấn luyện để đào tạo mô hình. Quá trình này bao gồm cập nhật trọng số của mô hình sao cho nó có khả năng dự đoán đúng trên dữ liệu đào tạo.

6. Đánh Giá Mô Hình: Sử dụng tập kiểm thử để đánh giá hiệu suất của mô hình. Các độ đo như độ chính xác, độ phủ, và ma trận nhầm lẫn có thể được sử dụng để đánh giá khả năng dự đoán của mô hình.

7. Tinh Chỉnh Mô Hình: Nếu cần thiết, điều chỉnh các siêu tham số của mô hình dựa trên tập validation để cải thiện hiệu suất.

8. Triển Khai Mô Hình: Khi mô hình đã được huấn luyện và đánh giá, triển khai nó vào môi trường thực tế để sử dụng trong ứng dụng hoặc quy trình kinh doanh.

9. Theo Dõi và Tối Ưu Hóa: Theo dõi hiệu suất của mô hình trong môi trường sản xuất và thực hiện tối ưu hóa nếu cần thiết. Cập nhật mô hình khi có dữ liệu mới và nhu cầu cụ thể.

2.1.3 Các chức năng cụ thể của học máy

- Mô tả khái niệm (concept description): thiên về mô tả, tổng hợp và tóm tắt khái niệm. Ví dụ: tóm tắt văn bản.

- Phân lớp và dự đoán (classification & prediction): xếp một đối tượng vào một trong những lớp đã biết trước. Ví dụ: phân lớp vùng địa lý theo dữ liệu thời tiết. Hướng tiếp cận này thường sử dụng một số kỹ thuật của machine learning như cây quyết định (decision tree), mạng nơ-ron nhân tạo (neural network), .v.v. Người ta còn gọi phân lớp là học có giám sát (học có thầy).

- Phân cụm (clustering): xếp các đối tượng theo từng cụm (số lượng cũng như tên của cụm chưa được biết trước. Người ta còn gọi phân cụm là học không giám sát (học không thầy).

- Luật kết hợp (association rules): là dạng luật biểu diễn tri thức ở dạng khá đơn giản. Ví dụ: “60 % nam giới vào siêu thị nếu mua bia thì có tới 80% trong số họ sẽ mua thêm thịt bò khô”. Luật kết hợp được ứng dụng nhiều trong lĩnh vực kinh doanh, y học, tin-sinh, tài chính & thị trường chứng khoán, .v.v.

- Regression – Outline (Hồi quy): dự đoán một giá trị số bằng cách sử dụng các biến đã biết mà không cần phải tìm ra mối quan hệ giữa các biến đó.

2.1.4 Các ứng dụng của học máy

Ứng dụng: Học máy có ứng dụng rộng khắp trong các ngành khoa học/sản xuất, đặc biệt những ngành cần phân tích khối lượng dữ liệu khổng lồ. Một số ứng dụng thường thấy

Nhận Diện Ảnh: Ứng Dụng: Nhận diện khuôn mặt, phân loại đối tượng trong ảnh, nhận dạng biển số xe, phát hiện vật thể trong video, và tạo mô tả tự động cho hình ảnh.

Ngôn Ngữ Tự Nhiên và Xử Lý Ngôn Ngữ Tự Nhiên (NLP): Ứng Dụng: Dịch ngôn ngữ tự nhiên, phân loại cảm xúc từ văn bản, tạo ra mô hình ngôn ngữ tự nhiên, và tạo ra chatbot có khả năng trò chuyện thông minh.

Hệ Thống Tư Vấn và Gợi Ý: Ứng Dụng: Gợi ý sản phẩm dựa trên lịch sử mua sắm, tư vấn về sức khỏe và lối sống dựa trên dữ liệu cá nhân, và tư vấn tài chính cá nhân.

Quảng Cáo và Tiếp Thị: Ứng Dụng: Tối ưu hóa chiến lược quảng cáo, phân loại khách hàng tiềm năng, và tùy chỉnh nội dung quảng cáo dựa trên hành vi người dùng.

Y Tế và Chăm Sóc Sức Khỏe: Ứng Dụng: Dự đoán và chẩn đoán bệnh, tìm kiếm phác đồ điều trị tối ưu, theo dõi dữ liệu sức khỏe cá nhân, và tối ưu hóa quy trình y tế.

Tài Chính và Giao Dịch Tài Chính: Ứng Dụng: Dự báo giá chứng khoán, phân loại gian lận tài khoản, quản lý rủi ro tín dụng, và tư vấn đầu tư.

Ô Tô Tự Hành và Giao Thông Thông Minh: Ứng Dụng: Nhận diện vật thể trên đường, dự đoán lưu lượng giao thông, và tối ưu hóa tuyến đường cho ô tô tự hành.

Chẩn Đoán Y Khoa và Hỗ Trợ Quyết Định: Ứng Dụng: Phân loại hình ảnh y khoa như X-quang và MRI, dự đoán nguy cơ bệnh tật, và hỗ trợ quyết định trong điều trị.

Quản Lý Chuỗi Cung Ứng: Ứng Dụng: Dự đoán nhu cầu nguồn cung, tối ưu hóa quá trình sản xuất và lưu kho, và giảm thiểu rủi ro trong chuỗi cung ứng.

Giáo Dục: Ứng Dụng: Tùy chỉnh nội dung giảng dạy, đánh giá sinh viên dựa trên hiệu suất cá nhân, và phát triển hệ thống học tập cá nhân hóa.

2.2 Data Preprocessing(Tiền xử lý dữ liệu)

Tiền xử lý dữ liệu là một bước quan trọng trong quy trình phân tích dữ liệu và học máy. Nó liên quan đến việc làm sạch và chuyển đổi dữ liệu thô thành định dạng có thể được sử dụng hiệu quả cho việc lập mô hình. Dưới đây là hướng dẫn toàn diện về các kỹ thuật và bước tiền xử lý dữ liệu khác nhau:

1. **Thu thập dữ liệu:**

- Thu thập dữ liệu từ nhiều nguồn khác nhau, đảm bảo dữ liệu phù hợp với vấn đề bạn muốn giải quyết.

- Xem xét chất lượng, số lượng và mức độ phù hợp của dữ liệu.

2. **Kiểm tra dữ liệu:**

- Thực hiện phân tích dữ liệu khám phá (EDA) để hiểu cấu trúc, loại và cách phân bố các tính năng trong tập dữ liệu của bạn.

- Xác định bất kỳ sự bất thường, giá trị bị thiếu hoặc ngoại lệ.

Số lượng phiên bản và tính năng: Kiểm tra kích thước của tập dữ liệu về số lượng hàng (phiên bản) và cột (tính năng).

Loại dữ liệu: Xác định loại dữ liệu của từng tính năng (ví dụ: số, phân loại, văn bản).

Thông kê Tóm tắt: Tính toán các số liệu thống kê cơ bản như giá trị trung bình, trung vị, độ lệch chuẩn, tối thiểu và tối đa cho các đặc điểm số.

Trực quan hóa phân phối: Vẽ biểu đồ hoặc biểu đồ mật độ hạt nhân để trực quan hóa sự phân bố của các tính năng số.

3. **Xử lý dữ liệu bị thiếu:**

- Xác định và xử lý các giá trị còn thiếu thông qua các phương pháp như thay thế hoặc loại bỏ các trường hợp bị ảnh hưởng.

Sử dụng Thông kê Mô tả: Kiểm tra các giá trị còn thiếu bằng cách sử dụng số liệu thống kê tóm tắt. Các hàm như `info()` trong thư viện Pandas của Python hoặc `summary()` trong R có thể cung cấp cái nhìn tổng quan nhanh chóng.

Kiểm tra trực quan: Tạo trực quan hóa như bản đồ nhiệt hoặc biểu đồ thanh để xác định trực quan các giá trị còn thiếu.

Phân tích trường hợp hoàn chỉnh (CCA): Xóa các hàng chứa bất kỳ giá trị bị thiếu nào. Điều này phù hợp khi dữ liệu bị thiếu bị hạn chế và việc xóa các hàng không ảnh hưởng đáng kể đến việc phân tích.

Bỏ cột: Nếu một đối tượng có tỷ lệ giá trị bị thiếu cao, hãy cân nhắc việc bỏ toàn bộ cột.

- Xem xét tác động của việc thiếu dữ liệu trên toàn bộ tập dữ liệu.

4. **Làm sạch dữ liệu:**

- Giải quyết sự không nhất quán, lỗi và các ngoại lệ trong tập dữ liệu.

- Loại bỏ các thông tin trùng lặp và không liên quan không đóng góp cho nhiệm vụ lập mô hình.

Xử lý dữ liệu trùng lặp:

Xác định các hàng trùng lặp: Sử dụng các phương thức như `trùng_lặp()` trong Pandas của Python hoặc `trùng_lặp()` trong R để xác định và đánh dấu các hàng trùng lặp.

Xóa các hàng trùng lặp: Loại bỏ các hàng trùng lặp trong khi giữ lại lần xuất hiện đầu tiên hoặc dựa trên các tiêu chí cụ thể.

Xử lý các ngoại lệ:

Kiểm tra trực quan: Sử dụng biểu đồ hộp, biểu đồ phân tán hoặc biểu đồ để xác định các giá trị ngoại lệ.

Phương pháp thống kê: Sử dụng các kỹ thuật thống kê như điểm Z hoặc IQR để phát hiện và xử lý các ngoại lệ.

Cắt xén hoặc Winsorizing: Cắt bớt các giá trị cực trị hoặc thay thế chúng bằng các giá trị cực trị ít hơn.

Xử lý dữ liệu không nhất quán:

Kiểm tra tính nhất quán: Xác định sự không nhất quán trong dữ liệu phân loại hoặc dữ liệu số phải tuân theo các quy tắc nhất định.

Nhận dạng mẫu: Sử dụng biểu thức thông thường hoặc khớp chuỗi để xác định các mẫu không nhất quán trong dữ liệu văn bản.

Sửa các kiểu dữ liệu:

Đảm bảo loại dữ liệu chính xác: Xác minh rằng mỗi biến có loại dữ liệu thích hợp (ví dụ: số, phân loại, ngày giờ).

Chuyển đổi kiểu dữ liệu: Sử dụng các hàm như `astype()` trong Pandas để chuyển đổi kiểu dữ liệu nếu cần.

Xử lý dữ liệu không đầy đủ:

Phép tính: Giải quyết các giá trị còn thiếu thông qua các phương pháp tính toán như giá trị trung bình, trung vị, chế độ hoặc các kỹ thuật nâng cao như phép tính hồi quy.

Cân nhắc: Chọn chiến lược áp dụng dựa trên tính chất và phân bố dữ liệu bị thiếu.

Tiêu chuẩn hóa và chuẩn hóa:

Dữ liệu số theo tỷ lệ: Chuẩn hóa các đặc điểm số để có giá trị trung bình bằng 0 và độ lệch chuẩn là 1.

Chuẩn hóa: Chuẩn hóa dữ liệu số thành một phạm vi cụ thể, chẳng hạn như [0, 1].

Xử lý các định dạng ngày và giờ không nhất quán:

Phân tích ngày tháng: Đảm bảo tính nhất quán trong định dạng ngày và giờ bằng cách sử dụng các hàm như `pd.to_datetime()` trong Pandas.

Chuẩn hóa các định dạng: Chuyển đổi định dạng ngày và giờ thành một cách trình bày nhất quán.

Xử lý dữ liệu không liên quan:

Lựa chọn tính năng: Xác định và loại bỏ các tính năng không liên quan không đóng góp đáng kể cho nhiệm vụ phân tích hoặc mô hình hóa.

Lọc: Loại trừ các bản ghi không liên quan dựa trên các tiêu chí cụ thể.

Xử lý các cân không nhất quán:

Tiêu chuẩn hóa thang đo: Chuẩn hóa các đặc điểm số với các thang đo khác nhau để đảm bảo so sánh công bằng.

Sử dụng Robust Scaler: Áp dụng các phương pháp chia tỷ lệ mạnh mẽ để xử lý các giá trị ngoại lệ.

Xử lý tập dữ liệu lớn:

Lấy mẫu: Khi xử lý các tập dữ liệu lớn, hãy cân nhắc lấy mẫu ngẫu nhiên để phân tích dữ liệu thăm dò và làm sạch ban đầu.

Xử lý song song: Khám phá các kỹ thuật xử lý song song để thực hiện các thao tác làm sạch nhanh hơn.

5. ****Kỹ thuật tính năng:****

- Tạo các tính năng mới hoặc chuyển đổi các tính năng hiện có để nâng cao hiệu suất của mô hình.

- Chuẩn hóa hoặc chia tỷ lệ các đặc điểm số để đảm bảo chúng ở cùng một tỷ lệ.

6. ****Xử lý dữ liệu phân loại:****

- Chuyển đổi các biến phân loại thành biểu diễn số thông qua các kỹ thuật như mã hóa một nóng hoặc mã hóa nhãn.

- Xem xét việc sử dụng mã hóa mục tiêu cho các đặc điểm phân loại có số lượng bản số cao.

7. ****Chia tách dữ liệu:****

- Chia tập dữ liệu thành các tập huấn luyện, xác nhận và kiểm tra để đánh giá hiệu suất mô hình một cách hiệu quả.

- Đảm bảo rằng việc phân chia duy trì sự phân phối của biến mục tiêu.

8. ****Xử lý các lớp mất cân bằng:****

- Giải quyết sự mất cân bằng của lớp bằng cách lấy mẫu quá mức cho lớp thiểu số, lấy mẫu dưới lớp đa số hoặc sử dụng các kỹ thuật tạo dữ liệu tổng hợp.

- Cân nhắc sử dụng các thước đo đánh giá thích hợp cho các tập dữ liệu mất cân bằng.

9. ****Chuẩn hóa và tiêu chuẩn hóa:****

- Chuẩn hóa các đặc tính số về thang đo chuẩn, chẳng hạn như [0, 1].

- Chuẩn hóa các đặc trưng có giá trị trung bình bằng 0 và độ lệch chuẩn bằng 1.

10. ****Xử lý dữ liệu chuỗi thời gian:****

- Nếu xử lý dữ liệu chuỗi thời gian, hãy xem xét các tính năng độ trễ, thống kê cuộn hoặc lấy mẫu lại để nắm bắt các mẫu thời gian một cách hiệu quả.

- Đảm bảo thứ tự thời gian của các điểm dữ liệu.

11. ****Xử lý dữ liệu văn bản:****

- Xử lý trước dữ liệu văn bản bằng cách loại bỏ các từ dừng, bắt nguồn và chuyển đổi văn bản thành dạng biểu diễn số bằng cách sử dụng các kỹ thuật như TF-IDF hoặc nhúng từ.

- Cân nhắc sử dụng các kỹ thuật xử lý ngôn ngữ tự nhiên (NLP) để xử lý trước văn bản nâng cao.

12. ****Xử lý dữ liệu ồn ào:****

- Xử lý nhiễu trong tập dữ liệu thông qua kỹ thuật lọc hoặc làm mịn, tùy theo tính chất của nhiễu.

- Sử dụng các phương pháp thống kê mạnh mẽ để giảm tác động của các ngoại lệ.

13. ****Giảm kích thước:****

- Sử dụng các kỹ thuật như Phân tích thành phần chính (PCA) hoặc Nhúng hàng xóm ngẫu nhiên phân phối t (t-SNE) để giảm tính chiều của tập dữ liệu.

- Đánh giá phương sai giải thích để xác định số lượng thành phần tối ưu.

14. ****Mã hóa dữ liệu:****

- Mã hóa dữ liệu thứ tự bằng cách sử dụng các kỹ thuật bảo toàn thứ tự, chẳng hạn như mã hóa thứ tự.
- Cân nhắc sử dụng mã hóa nhị phân hoặc mã hóa băm cho các đặc điểm phân loại có số lượng cao.

15. ****Chia tỷ lệ dữ liệu:****

- Áp dụng tỷ lệ cho các đối tượng số, đặc biệt trong các thuật toán nhạy cảm với độ lớn của đối tượng (ví dụ: thuật toán dựa trên độ dốc).
- Đảm bảo rằng việc chia tỷ lệ được thực hiện riêng biệt cho các tập dữ liệu huấn luyện và kiểm tra.

16. ****Tăng cường dữ liệu (đối với dữ liệu hình ảnh):****

- Giới thiệu các biến thể trong dữ liệu huấn luyện thông qua các kỹ thuật như xoay, lật hoặc thu phóng, đặc biệt hữu ích cho các tác vụ phân loại hình ảnh.
- Sử dụng các thư viện mở rộng để tạo ra các mẫu đào tạo đa dạng.

17. ****Tài liệu:****

- Ghi lại tất cả các bước tiền xử lý, bao gồm mọi biến đổi hoặc thao tác được áp dụng cho dữ liệu.
- Duy trì một bản ghi rõ ràng về quy trình tiền xử lý để có thể tái tạo.

18. ****Xác thực và Xác thực chéo:****

- Sử dụng các bộ xác thực hoặc xác thực chéo để đánh giá tác động của các lựa chọn tiền xử lý đối với việc khái quát hóa mô hình.
- Tránh rò rỉ dữ liệu bằng cách thực hiện tiền xử lý trong mỗi lần xác thực chéo.

19. ****Kiểm tra độ tin cậy:****

- Kiểm tra độ chắc chắn của quy trình tiền xử lý của bạn bằng cách đưa ra các lỗi tổng hợp hoặc đánh giá dựa trên dữ liệu ngoài mẫu.
- Tiến hành phân tích độ nhạy để đánh giá tác động của các lựa chọn tiền xử lý đến hiệu suất của mô hình.

20. ****Triển khai quy trình:****

- Triển khai quy trình tiền xử lý để đảm bảo tính nhất quán và khả năng tái tạo, đặc biệt khi triển khai các mô hình trong các ứng dụng trong thế giới thực.
- Tích hợp quy trình tiền xử lý vào quy trình học máy tổng thể.

2.3 Thuật toán

2.3.1 Decision Tree

Một cây quyết định là một cấu trúc thuật toán giống như flowchart, với mỗi một nút (node) là một phép so sánh, nhằm thực hiện thử một đặc trưng nào đó. Mỗi một nhánh (branch, edge, etc.) thể hiện kết quả của phép so sánh đó, và mỗi một ngọn, tức nút cuối (leaf node), biểu diễn kết quả phân loại của thuật toán, sau khi tính. Đường đi từ gốc đến ngọn thể hiện luật phân loại (classification rules).

Mục tiêu chính là xây dựng được một mô hình có khả năng dự đoán giá trị của đầu ra dựa vào một vài biến đầu vào.

Cây quyết định là một mô hình đơn giản để ta phân loại dữ liệu. Chúng ta giả sử rằng các đặc trưng của dữ liệu đầu vào thuộc các miền rời rạc có hạn, và đầu ra chỉ là một biến đơn, gọi là lớp đoán nhận. Mỗi một phần tử trong miền đoán nhận được gọi là một lớp. Một cây đoán nhận hoặc một cây phân loại là cây có các nút không phải lá được đánh nhãn với một đặc trưng của dữ liệu đầu vào. Các nhánh con từ các nút không phải lá được đánh dấu bằng các giá trị có thể đạt được của nút đó, hoặc dẫn tới một nút không phải lá đại diện cho đặc trưng khác. Các nút lá của cây được đánh dấu bằng nhãn của dữ liệu, hoặc xác suất cụ thể dữ liệu sẽ rơi vào nhãn đó. Nếu cây quyết định được xây tốt, cây đó sẽ bị nghiêng về một tập con các lớp cụ thể.

Cây được xây bằng cách phân chia nhiều lần tập đặc trưng lớn nhất, tạo nên gốc của cây, thành các tập con, tạo nên những cây con. Quá trình phân chia được tạo thông qua các quy tắc dựa vào những đặc trưng, thường sẽ theo quy tắc sau:

Ta bắt đầu từ một cây với một nút lá duy nhất, tức gốc của cây, và gán cho lá này một nhãn, dựa trên luật đa số giữa các nhãn trên toàn bộ tập huấn luyện. Sau đó, ta thực hiện một loạt các bước tiến. Với mỗi bước tiến, ta xem xét các hệ quả khi phân nhánh. Ta định nghĩa một hàm tăng trưởng (hoặc hàm mất mát), nhằm đo sự tăng trưởng của hiệu suất mô hình sau khi phân nhánh. Trong các trường hợp phân nhánh khác nhau, ta chọn cách phân nhánh giúp ta tối đa hóa tăng trưởng hiệu suất mô hình, hoặc có thể chọn không phân nhánh.

Quá trình phân chia trên các tập con kế tiếp được thực hiện theo kiểu đệ quy được gọi là phân hoạch đệ quy (recursive partitioning). Quá trình đệ quy hoàn thành khi các lá đã bao phủ toàn bộ các đặc trưng, hoặc khi quá trình phân chia không còn thêm hiệu suất cho mô hình. Quá trình xây dựng cây theo quy nạp từ trên xuống (top-down induction of decision trees - viết tắt: TDIDT) là giải thuật tham lam (greedy algorithm), và là chiến thuật thông dụng nhất để huấn luyện cây quyết định từ dữ liệu.

Trong khai phá dữ liệu, các cây quyết định được miêu tả thông qua các kỹ thuật toán học và kỹ thuật tính toán để giúp mô tả, phân loại và khái quát hóa dữ liệu.

Dữ liệu thường được tổ chức dưới dạng:

$$(x, Y) = (x_1, x_2, x_3, \dots, x_k, Y)$$

Biến phụ thuộc, Y chính là nhãn, hay là đầu ra mong muốn mà ta cần phải học, phân loại và khái quát hóa. Vector x được tạo bởi các đặc trưng, $x_1, x_2, x_3, \dots$, được sử dụng cho mục đích trên.

Cây quyết định trong khai phá dữ liệu có 2 kiểu:

Cây phân loại là cây có giá trị đầu ra là các lớp rời rạc của dữ liệu.

Cây hồi quy là cây có giá trị đầu ra là các giá trị thực

Thuật toán CART, cây phân loại và hồi quy (classification and regression tree), là thuật ngữ nói chung để ám chỉ bất kỳ 2 loại cây trên, được Breiman và đồng nghiệp sử dụng lần đầu tiên năm 1984. Cây hồi quy và cây phân loại có một số điểm tương đồng cũng như khác biệt, ví dụ như luật để quyết định phân nhánh.

Một số kỹ thuật khác, thường được gọi là các kỹ thuật học kết hợp (ensemble methods), xây dựng nhiều cây cùng một lúc:

Boosted trees: xây dựng một tập thể bằng cách huấn luyện từng cây riêng lẻ để hỗ trợ các cây sai lệch trước đó.

Bootstrap aggregated: xây dựng nhiều cây quyết định cùng một lúc bằng cách liên tục tái lấy mẫu và thay thế các tập dữ liệu con, và thực hiện biểu quyết để quyết định nhãn đầu ra. Một ví dụ điển hình chính là Random Forest.

Rotation forest: xây dựng nhiều cây quyết định, với mỗi cây quyết định được huấn luyện trên các tập dữ liệu con với các đặc trưng được lựa chọn ngẫu nhiên. Trước khi huấn luyện, các tập dữ liệu con được thực hiện PCA.

Các thuật toán nổi tiếng gồm:

ID3 (Iterative Dichotomiser 3)

C4.5

CART

Chi-square automatic interaction detection

MARS

Cây quy nạp điều kiện

ID3 và CART được phát minh độc lập trong cùng một khoảng thời gian (giữa 1970 -> 1980), nhưng cùng một hướng tiếp cận để huấn luyện cây quyết định từ các bộ dữ liệu.

Các phép đo để đánh giá cây quyết định:

Ước lượng số kết quả dương tính chính xác (Estimate of Positive Correctness):

Một phương pháp đánh giá độ chính xác đơn giản chính là tính độ lệch giữa giá trị dương tính thật (True Positive) và giá trị dương tính giả (False Positive): $E_p = TP - EP$

Tiêu chí phân tách (Selection criterion)

Độ thuần khiết Gini (hay còn gọi là độ vẩn đục Gini) và Độ tăng dữ liệu (còn gọi là độ lợi dữ liệu) là 2 tiêu chí phân tách quan trọng để hỗ trợ Cây quyết định phân nhánh.

Độ thuần khiết Gini (Gini impurity, khác với hệ số Gini - Gini index)

Độ thuần khiết Gini được sử dụng bởi thuật toán CART, là độ đo tần suất một phần tử được chọn ngẫu nhiên từ tập hợp bị dán nhãn sai nếu nó được dán nhãn ngẫu nhiên theo phân phối nhãn trong tập dữ liệu con.

Độ thuần khiết Gini được tính bằng cách lấy tổng xác suất p_i của bản ghi i được chọn nhãn với xác suất $\sum_{k \neq i} p_k = 1 - p_i$, tức xác suất khi phân loại sai bản ghi i . Độ thuần khiết đạt giá trị nhỏ nhất, tức bằng 0 khi tất cả các trường hợp tại điểm nút cuối đều rơi vào cùng một giá trị. Độ thuần khiết Gini cũng là độ đo lý thuyết thông tin và tương ứng với entropy dạng Tsallis với hệ số biến dạng $q=2$, trong Vật lý, công thức entropy này đại diện cho tính mất thông tin trong các hệ thống lượng tử mất cân bằng, tắt dần.

Khi có $q \rightarrow 1$, công thức trở về dạng Boltzmann - Gibbs hoặc công thức entropy của Shannon. Với các điều kiện này, độ thuần khiết Gini có tác dụng như một biến thể của phép đo entropy cho các cây quyết định.

Để tính độ thuần khiết Gini cho một tập các điểm dữ liệu có J lớp, với $i \in \{1, 2, \dots, J\}$, và để p_i là lượng bản ghi được đánh dấu với lớp i trong tập hợp, thì:

$$I_g(p) = \sum_{i=1}^J (p_i \sum_{k \neq i} p_k) = \sum_{i=1}^J (p_i - p_i^2) = \sum_{i=1}^J p_i - \sum_{i=1}^J p_i^2 = 1 - \sum_{i=1}^J p_i^2$$

Độ tăng thông tin (Information gain)

Độ tăng thông tin dựa vào khái niệm entropy và khái niệm nội dung thông tin trong lý thuyết thông tin.

Định nghĩa của entropy:

$$H(T) = I_E(p_1, p_2, \dots, p_J) = - \sum_{i=1}^J p_i \log_2 p_i$$

trong đó, $p_1, p_2, \dots$ là các xác suất dữ liệu rơi vào từng lớp tại nút con xuất phát từ phân nhánh của cây.

$$\begin{aligned} \overbrace{IG(T, a)}^{\text{information gain}} &= \overbrace{H(T)}^{\text{entropy (parent)}} - \overbrace{H(T | a)}^{\text{sum of entropies (children)}} \\ &= - \sum_{i=1}^J p_i \log_2 p_i - \left[\sum_{i=1}^J - \Pr(i | a) \log_2 \Pr(i | a) \right] \end{aligned}$$

Ta sau đó lấy trung bình của các giá trị có thể xảy ra của A

$$\begin{aligned} \overbrace{E_A(IG(T, a))}^{\text{expected information gain}} &= \overbrace{I(T; A)}^{\text{mutual information between } T \text{ and } A} = \overbrace{H(T)}^{\text{entropy (parent)}} - \overbrace{H(T | A)}^{\text{weighted sum of entropies (children)}} \\ &= - \sum_{i=1}^J p_i \log_2 p_i - \sum_a p(a) \sum_{i=1}^J - \Pr(i | a) \log_2 \Pr(i | a) \end{aligned}$$

Ta tính tổng các entropy có trọng số thông qua công thức sau:

$$H(T | A) = \sum_a p(a) \sum_{i=1}^J -\Pr(i | a) \log_2 \Pr(i | a)$$

Tức, lượng thông tin tăng lên theo dự kiến chính là lượng thông tin tương quan (mutual information). Tính trung bình thì lượng entropy T giảm đi cũng chính là lượng thông tin tương quan.

Lượng thông tin tăng lên chính là tiêu chí để ta quyết định cây phân nhánh dựa trên đặc trưng nào. Tại mỗi điểm phân nhánh, ta nên chọn cách phân nhánh để có thể tạo ra các nút con nhất quán nhất. Để xây dựng cây, ta cần tính độ tăng thông tin của các cách phân nhánh có thể xảy ra. Cách phân nhánh tốt nhất chính là cách tối ưu độ tăng thông tin nhất. Quá trình này được lặp lại nhiều lần với các nút không lá cho đến khi cây hoàn thiện.

Độ thuần khiết Gini và độ tăng thông tin là hai độ đo thông tin để quyết định phân nhánh. Thông thường, sử dụng một trong hai độ đo này sẽ không tạo ra hiệu suất quá khác biệt. Thường, độ thuần khiết Gini sẽ nhanh hơn, nên được sử dụng nhiều hơn. Và thông thường, cây tạo ra bởi 2 độ đo này cũng tương tự nhau. Độ thuần khiết Gini sẽ cho ta cây với các nhánh cô lập theo phân phối dữ liệu, trong khi đó thì độ tăng thông tin cho ta cây cân bằng hơn.

2.3.2 Naive Bayes

Naive Bayes là một thuật toán phân lớp được mô hình hoá dựa trên định lý Bayes trong xác suất thống kê:

$$P(y|X) = \frac{P(X|y)P(y)}{P(X)}$$

, trong đó:

$P(y|X)$ gọi là posterior probability: xác suất của mục tiêu y với điều kiện có đặc trưng X

$P(X|y)$ gọi là likelihood: xác suất của đặc trưng X khi đã biết mục tiêu y

$P(y)$ gọi là prior probability của mục tiêu y

$P(X)$ gọi là prior probability của đặc trưng X

Ở đây, X là vector các đặc trưng, có thể viết dưới dạng:

$$X = (x_1, x_2, x_3, \dots, x_n)$$

Khi đó, đẳng thức Bayes trở thành:

$$P(y|x_1, \dots, x_n) = \frac{P(x_1|y)P(x_2|y)\dots P(x_n|y)P(y)}{P(x_1)P(x_2)\dots P(x_n)}$$

Trong mô hình Naive Bayes, có hai giả thiết được đặt ra:

Các đặc trưng đưa vào mô hình là độc lập với nhau. Tức là sự thay đổi giá trị của một đặc trưng không ảnh hưởng đến các đặc trưng còn lại.

Các đặc trưng đưa vào mô hình có ảnh hưởng ngang nhau đối với đầu ra mục tiêu.

Khi đó, kết quả mục tiêu y để $P(y|X)$ đạt cực đại trở thành:

$$y = \operatorname{argmax}_y P(y) \prod_{i=1}^n P(x_i|y)$$

Chính vì hai giả thiết gần như không tồn tại trong thực tế trên, mô hình này mới được gọi là naive (ngây thơ). Tuy nhiên, chính sự đơn giản của nó với việc dự đoán rất nhanh kết quả đầu ra khiến nó được sử dụng rất nhiều trong thực tế trên những bộ dữ liệu lớn, đem lại kết quả khả quan. Một vài ứng dụng của Naive Bayes có thể kể đến như: lọc thư rác, phân loại văn bản, dự đoán sắc thái văn bản, ...

Một số kiểu mô hình Naive Bayes

+ Multinomial Naive Bayes

Mô hình này chủ yếu được sử dụng trong phân loại văn bản. Đặc trưng đầu vào ở đây chính là tần suất xuất hiện của từ trong văn bản đó.

+ Bernoulli Naive Bayes

Mô hình này được sử dụng khi các đặc trưng đầu vào chỉ nhận giá trị nhị phân 0 hoặc 1 (phân bố Bernoulli).

+ Gaussian Naive Bayes

Khi các đặc trưng nhận giá trị liên tục, ta giả sử các đặc trưng đó có phân phối Gaussian. Khi đó, likelihood sẽ có dạng:

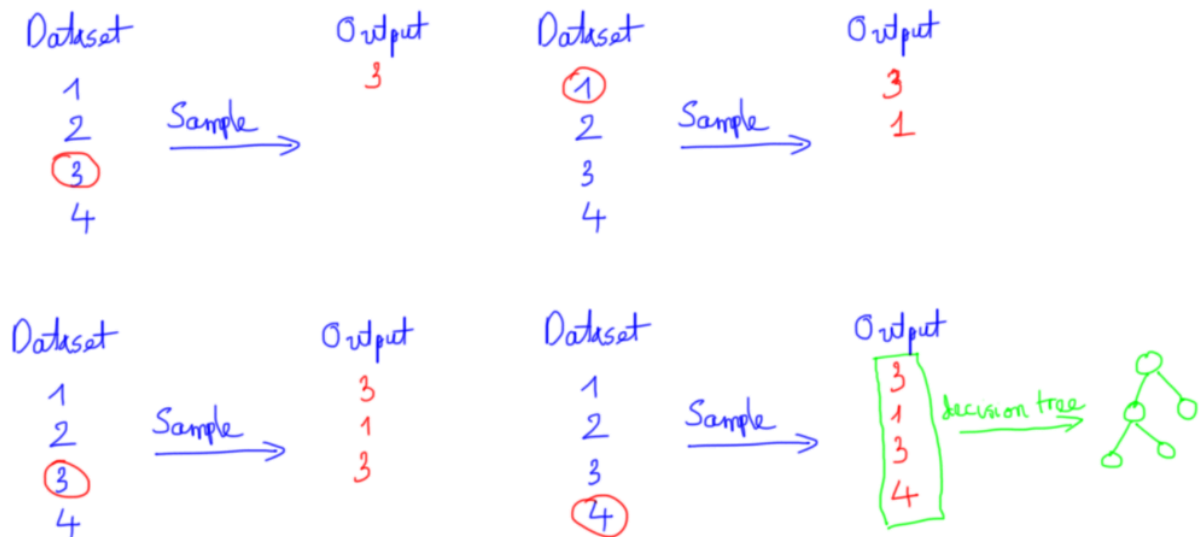
$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}\right)$$

2.3.3 Random Forest

Giả sử bộ dữ liệu của mình có n dữ liệu (sample) và mỗi dữ liệu có d thuộc tính (feature).

Để xây dựng mỗi cây quyết định mình sẽ làm như sau:

Lấy ngẫu nhiên n dữ liệu từ bộ dữ liệu với kỹ thuật Bootstrapping, hay còn gọi là random sampling with replacement. Tức khi mình sample được 1 dữ liệu thì mình không bỏ dữ liệu đấy ra mà vẫn giữ lại trong tập dữ liệu ban đầu, rồi tiếp tục sample cho tới khi sample đủ n dữ liệu. Khi dùng kỹ thuật này thì tập n dữ liệu mới của mình có thể có những dữ liệu bị trùng nhau.



Sau khi sample được n dữ liệu từ bước 1 thì mình chọn ngẫu nhiên ở k thuộc tính ($k < n$). Giờ mình được bộ dữ liệu mới gồm n dữ liệu và mỗi dữ liệu có k thuộc tính.

Dùng thuật toán Decision Tree để xây dựng cây quyết định với bộ dữ liệu ở bước 2.

Do quá trình xây dựng mỗi cây quyết định đều có yếu tố ngẫu nhiên (random) nên kết quả là các cây quyết định trong thuật toán Random Forest có thể khác nhau.

Thuật toán Random Forest sẽ bao gồm nhiều cây quyết định, mỗi cây được xây dựng dùng thuật toán Decision Tree trên tập dữ liệu khác nhau và dùng tập thuộc tính khác nhau. Sau đó kết quả dự đoán của thuật toán Random Forest sẽ được tổng hợp từ các cây quyết định.

Khi dùng thuật toán Random Forest, mình hay để ý các thuộc tính như: số lượng cây quyết định sẽ xây dựng, số lượng thuộc tính dùng để xây dựng cây. Ngoài ra, vẫn có các thuộc tính của thuật toán Decision Tree để xây dựng cây như độ sâu tối đa, số phần tử tối thiểu trong 1 node để có thể tách.

* Tại sao thuật toán Random Forest tốt

Trong thuật toán Decision Tree, khi xây dựng cây quyết định nếu để độ sâu tùy ý thì cây sẽ phân loại đúng hết các dữ liệu trong tập training dẫn đến mô hình có thể dự đoán tệ trên tập validation/test, khi đó mô hình bị overfitting, hay nói cách khác là mô hình có high variance.

Thuật toán Random Forest gồm nhiều cây quyết định, mỗi cây quyết định đều có những yếu tố ngẫu nhiên:

Lấy ngẫu nhiên dữ liệu để xây dựng cây quyết định.

Lấy ngẫu nhiên các thuộc tính để xây dựng cây quyết định.

Do mỗi cây quyết định trong thuật toán Random Forest không dùng tất cả dữ liệu training, cũng như không dùng tất cả các thuộc tính của dữ liệu để xây dựng cây nên mỗi cây có thể sẽ dự đoán không tốt, khi đó mỗi mô hình cây quyết định không bị overfitting mà có thể bị underfitting, hay nói cách khác là mô hình có high bias. Tuy nhiên, kết quả cuối cùng của thuật toán Random Forest lại tổng hợp từ nhiều cây quyết định, thế nên thông tin từ các cây sẽ bổ sung thông tin cho nhau, dẫn đến mô hình có low bias và low variance, hay mô hình có kết quả dự đoán tốt.

Ý tưởng tổng hợp các cây quyết định của thuật toán Random Forest giống với ý tưởng của The Wisdom of Crowds được đề xuất bởi by James Surowiecki vào năm 2004. The Wisdom of Crowds nói rằng thông thường tổng hợp thông tin từ 1 nhóm sẽ tốt hơn từ một cá nhân. Ở thuật toán Random Forest mình cũng tổng hợp thông tin từ 1 nhóm các cây quyết định và kết quả cho ra tốt hơn thuật toán Decision Tree với 1 cây quyết định.

Ví dụ: Mọi người muốn mua 1 sản phẩm trên tiki chẳng hạn, khi đọc review sản phẩm, nếu chỉ đọc 1 review thì có thể là ý kiến chủ quan của người đấy, hoặc sản phẩm người ấy mua không may bị lỗi gì; thông thường để có cái nhìn tốt về sản phẩm, mình hay đọc tất cả review rồi cho ra quyết định cuối cùng.

PHẦN 3: TRIỂN KHAI ĐỀ TÀI

3.1 Chuẩn bị dữ liệu

Môi trường, ngôn ngữ và các trang web hỗ trợ:

Ngôn ngữ: Python

Môi trường cài đặt: Google Colab <https://colab.research.google.com/>

Nguồn dữ liệu: Kaggle <https://www.kaggle.com/>

Công cụ hỗ trợ tra cứu, tham khảo: ChatGPT

Đầu tiên, tập dữ liệu được tải lên Google Colab và lưu trữ trong Google Drive để thuận tiện cho quá trình truy cập và xử lý. Do dữ liệu sử dụng trong đề tài là dữ liệu ảnh, thay vì đọc dữ liệu dưới dạng bảng bằng thư viện Pandas, nhóm sử dụng các công cụ xử lý ảnh của TensorFlow/Keras.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
import numpy as np
import warnings
warnings.filterwarnings("ignore", category=UserWarning)
import os
```

Hình 3.1.1 import thư viện

```
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

# ĐƯỜNG DẪN THƯ MỤC datadoan
data_dir = "/content/drive/MyDrive/datadoan"

# Mở / kiểm tra thư mục datadoan
print(" Nội dung thư mục datadoan:")
print(os.listdir(data_dir))
```

Hình 3.1.2 code đọc dữ liệu file

```
DATA_DIR = "/content/drive/MyDrive/datadoan/plant-health"
IMG_SIZE = (150, 150)
BATCH_SIZE = 32
VAL_SPLIT = 0.2
EPOCHS = 5
```

Hình 3.1.3 khai báo tham số

ATA_DIR

Biến DATA_DIR dùng để xác định đường dẫn đến thư mục chứa bộ dữ liệu ảnh. Thư mục này bao gồm các thư mục con tương ứng với từng lớp dữ liệu (ví dụ: *healthy* và *stressed*). Việc chỉ định đúng đường dẫn giúp mô hình có thể đọc và xử lý dữ liệu một cách chính xác.

IMG_SIZE = (150, 150)

Tham số này quy định kích thước chuẩn của ảnh đầu vào trước khi đưa vào mô hình. Tất cả hình ảnh trong bộ dữ liệu sẽ được resize về kích thước 150×150 pixel nhằm đảm bảo tính đồng nhất và giảm độ phức tạp tính toán khi huấn luyện mô hình.

BATCH_SIZE = 32

BATCH_SIZE xác định số lượng ảnh được đưa vào mô hình trong mỗi lần huấn luyện. Việc sử dụng batch giúp quá trình huấn luyện ổn định hơn, tối ưu bộ nhớ và cải thiện tốc độ học của mô hình.

VAL_SPLIT = 0.2

Tham số này dùng để chia dữ liệu thành hai tập: tập huấn luyện và tập kiểm tra (validation). Trong đó, 20% dữ liệu được sử dụng cho việc đánh giá mô hình trong quá trình huấn luyện, giúp theo dõi hiện tượng overfitting.

EPOCHS = 5

EPOCHS biểu thị số lần mô hình được huấn luyện lặp lại trên toàn bộ tập dữ liệu huấn luyện. Trong phạm vi bài tập môn học, nhóm lựa chọn số epoch nhỏ nhằm kiểm tra nhanh hiệu quả mô hình và tiết kiệm thời gian huấn luyện.

3.2 Kiểm tra và xử lý dữ liệu

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.2,
    horizontal_flip=True,
    validation_split=VAL_SPLIT
)
```

Hình 3.2.1 tiền xử lý và tăng cường dữ liệu

```
train_gen = train_datagen.flow_from_directory(
    directory=DATA_DIR,          # thư mục chứa các folder class
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='training',
    shuffle=True
)
```

Hình 3.2.2 Generator cho tập train

```
val_gen = train_datagen.flow_from_directory(
    directory=DATA_DIR,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    subset='validation',
    shuffle=False
)
```

HÌNH 3.2.3 Generator cho tập validation

```
class_names = list(train_gen.class_indices.keys())
num_classes = len(class_names)

print("Số lớp:", num_classes)
print("Các lớp:", class_names)
```

Hình 3.2.4 lấy tên các lớp

```
Found 566 images belonging to 2 classes.
Found 141 images belonging to 2 classes.
Số lớp: 2
Các lớp: ['healthy', 'unhealthy']
```

Hình 3.2.5

Sau khi sử dụng ImageDataGenerator để đọc dữ liệu từ thư mục và chia tập dữ liệu theo tỷ lệ huấn luyện và kiểm tra, chương trình thu được kết quả như sau:

-Tập huấn luyện (Training set): 566 ảnh

-Tập kiểm tra (Validation set): 141 ảnh

-Số lớp: 2

-Tên các lớp: healthy, unhealthy

Điều này cho thấy bộ dữ liệu được chia thành hai lớp rõ ràng, phù hợp cho bài toán phân loại ảnh nhị phân nhằm nhận diện tình trạng xà lách khỏe mạnh và xà lách bị stress.

```
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Input(shape=(IMG_SIZE, 3)), # (150,150,3)

    layers.Conv2D(32, (3,3), padding='same', activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(64, (3,3), padding='same', activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(128, (3,3), padding='same', activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Flatten(),

    layers.Dense(128, activation='relu'),
    layers.Dropout(0.4), # tăng nhẹ để chống overfitting

    layers.Dense(len(class_names), activation='softmax')
])

model.summary()
```

Hình 3.2.6 xây dựng mô hình CNN

Model: "sequential"		
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 150, 150, 32)	896
max_pooling2d (MaxPooling2D)	(None, 75, 75, 32)	0
conv2d_1 (Conv2D)	(None, 75, 75, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 37, 37, 64)	0
conv2d_2 (Conv2D)	(None, 37, 37, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 18, 18, 128)	0
flatten (Flatten)	(None, 41472)	0
dense (Dense)	(None, 128)	5,308,544
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 2)	258
Total params: 5,402,050 (20.61 MB)		
Trainable params: 5,402,050 (20.61 MB)		
Non-trainable params: 0 (0.00 B)		

Hình 3.2.6

Mô hình CNN được xây dựng gồm các lớp tích chập (Convolution), gộp cực đại (MaxPooling), lớp kết nối đầy đủ (Dense) và lớp Dropout nhằm giảm hiện tượng quá khớp. Theo kết quả `model.summary()`, mô hình có tổng cộng 5.402.050 tham số, trong đó toàn bộ đều là tham số có thể huấn luyện. Không có tham số cố định (non-trainable). Dung lượng mô hình khoảng 20.61 MB, phù hợp cho quá trình huấn luyện và thử nghiệm trên Google Colab.

```
model.compile(  
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),  
    loss='categorical_crossentropy',  
    metrics=['accuracy']  
)
```

3.2.7 Compile mô hình

```
history = model.fit(  
    train_gen,  
    validation_data=val_gen,  
    epochs=EPOCHS,  
    verbose=1  
)
```

Hình 3.2.8 Huấn luyện mô hình

```
Epoch 1/5  
18/18 ————— 51s 3s/step - accuracy: 0.5115 - loss: 0.9888 - val_accuracy: 0.5390 - val_loss: 0.6954  
Epoch 2/5  
18/18 ————— 49s 3s/step - accuracy: 0.6000 - loss: 0.6730 - val_accuracy: 0.5390 - val_loss: 0.7907  
Epoch 3/5  
18/18 ————— 79s 3s/step - accuracy: 0.5947 - loss: 0.6373 - val_accuracy: 0.4965 - val_loss: 0.7024  
Epoch 4/5  
18/18 ————— 46s 2s/step - accuracy: 0.7385 - loss: 0.5543 - val_accuracy: 0.5816 - val_loss: 0.6712  
Epoch 5/5  
18/18 ————— 46s 2s/step - accuracy: 0.7572 - loss: 0.4691 - val_accuracy: 0.6241 - val_loss: 0.6587
```

Hình 3.2.9

Mô hình được huấn luyện trong 5 epoch trên tập dữ liệu huấn luyện và đánh giá trên tập validation. Kết quả cho thấy hiệu suất mô hình được cải thiện dần theo từng epoch.

Epoch 1:

Độ chính xác huấn luyện đạt khoảng 51%, mô hình chưa học được nhiều đặc trưng, độ chính xác validation còn thấp.

Epoch 2 – 3:

Độ chính xác huấn luyện tăng nhẹ, tuy nhiên độ chính xác validation dao động và chưa ổn định, cho thấy mô hình vẫn đang trong quá trình học đặc trưng dữ liệu.

Epoch 4 – 5:

Mô hình bắt đầu hội tụ tốt hơn, accuracy trên tập huấn luyện tăng lên khoảng 75%,

trong khi accuracy trên tập validation đạt khoảng 62%, đồng thời giá trị loss giảm dần.

```
val_loss, val_acc = model.evaluate(val_gen, verbose=0)

print(f"\nĐộ chính xác trên tập validation: {val_acc:.4f}")
print(f"Giá trị loss trên tập validation: {val_loss:.4f}")
```

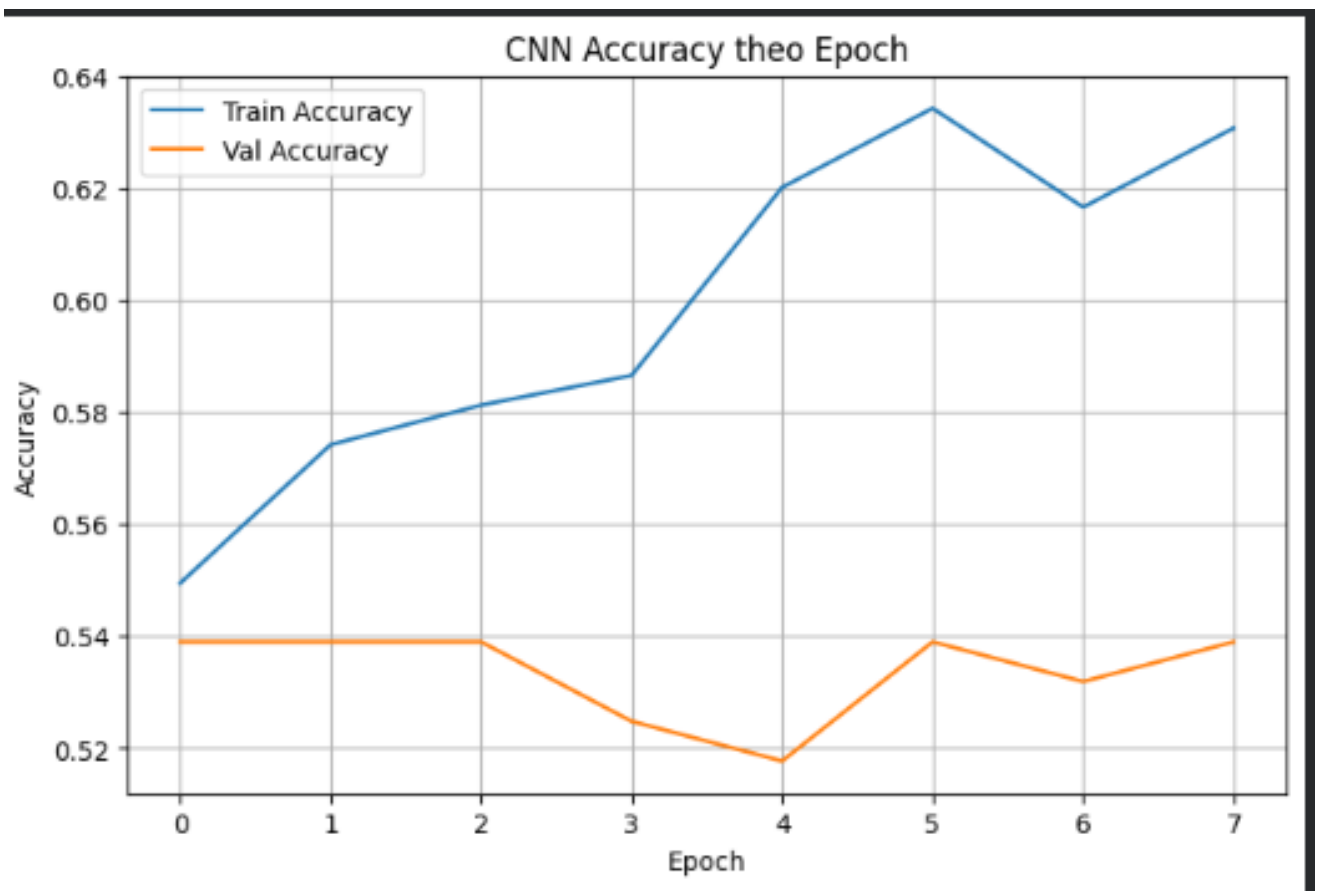
Hình 3.2.10 Đánh giá mô hình trên tập validation

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title('Biểu đồ độ chính xác qua các Epoch')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.show()
```

Hình 3.2.11 Biểu đồ Accuracy theo epoch



Hình 3.2.12

Độ chính xác trên tập huấn luyện tăng đều qua các epoch, cho thấy mô hình học tốt dữ liệu huấn luyện. Độ chính xác trên tập kiểm định có xu hướng tăng nhưng thấp hơn tập huấn luyện và dao động nhẹ. Khoảng cách giữa hai đường cho thấy mô hình có dấu hiệu overfitting nhẹ, cần tối ưu thêm để cải thiện khả năng tổng quát hóa.

```
x = []
y = []

class_names = sorted(os.listdir(DATA_DIR))

for label, class_name in enumerate(class_names):
    class_path = os.path.join(DATA_DIR, class_name)
    if not os.path.isdir(class_path):
        continue

    for img_name in os.listdir(class_path):
        img_path = os.path.join(class_path, img_name)

        img = cv2.imread(img_path)
        if img is None:
            continue

        img = cv2.resize(img, IMG_SIZE)
        img = img / 255.0      # chuẩn hoá giống CNN
        img = img.flatten()    # chuyển ảnh thành vector 1D

        x.append(img)
        y.append(label)

x = np.array(x)
y = np.array(y)

print("Naive Bayes - Shape x:", x.shape)
print("Naive Bayes - Shape y:", y.shape)
```

Hình 3.2.13 Load & chuẩn hoá dữ liệu ảnh cho Naive Bayes

```
x_train, x_val, y_train, y_val = train_test_split(
    x, y,
    test_size=VAL_SPLIT,
    random_state=42,
    stratify=y
)
```

Hình 3.2.14 Chia tập train

```
nb_model = GaussianNB()
nb_model.fit(x_train, y_train)
```

Hình 3.2.15 Huấn luyện mô hình Naive Bayes

```
y_pred = nb_model.predict(X_val)

nb_acc = accuracy_score(y_val, y_pred)
print("\nĐộ chính xác Naive Bayes:", nb_acc)

print("\nBáo cáo phân loại Naive Bayes:")
print(classification_report(y_val, y_pred, target_names=class_names))
```

Hình 3.2.16 Đánh giá mô hình Naive Bayes

```
Báo cáo phân loại Naive Bayes:
              precision    recall  f1-score   support

 healthy         0.60      0.74      0.66         65
 unhealthy       0.73      0.58      0.65         77

 accuracy              0.65         142
 macro avg           0.66      0.66      0.65         142
 weighted avg        0.67      0.65      0.65         142
```

Hình3.2.17

Mô hình Naive Bayes đạt độ chính xác 65%. Lớp *healthy* có recall cao (0.74) nhưng precision thấp hơn, trong khi lớp *unhealthy* có precision cao (0.73) nhưng recall thấp (0.58). Giá trị F1-score của hai lớp xấp xỉ nhau (~0.65), cho thấy mô hình phân loại ở mức trung bình và còn hạn chế trong việc cân bằng giữa hai lớp.

```
print("\nSO SÁNH MÔ HÌNH")
print(f"CNN Accuracy      : {val_acc:.4f}")
print(f"Naive Bayes Accuracy: {nb_acc:.4f}")
```

Hình 3.2.17 SO SÁNH CNN & NAIVE BAYES

```
SO SÁNH MÔ HÌNH
CNN Accuracy      : 0.6028
Naive Bayes Accuracy: 0.6549
```

Hình 3.2.18 kết quả so sánh

Mô hình Naive Bayes đạt độ chính xác 65.49%, cao hơn so với mô hình CNN (60.28%). Điều này cho thấy trong bài toán này, Naive Bayes cho kết quả phân loại tốt hơn CNN. CNN có thể chưa được tối ưu hoặc cần thêm dữ liệu huấn luyện để phát huy hiệu quả.

```
datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=30,
    zoom_range=0.2,
    horizontal_flip=True,
    validation_split=0.2
)
```

Hình 3.2.17 ImageDataGenerator

```
train_generator = datagen.flow_from_directory(
    directory=DATASET_PATH,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="binary",
    subset="training",
    shuffle=True
)
```

Hình 3.2.18 Train generator

```
val_generator = datagen.flow_from_directory(
    directory=DATASET_PATH,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="binary",
    subset="validation",
    shuffle=False
)
```

Hình 3.2.19 Validation generator

```
Found 566 images belonging to 1 classes.
Found 141 images belonging to 1 classes.
```

Hình 3.2.20

Sử dụng lớp ImageDataGenerator, nhóm thực hiện chuẩn hóa dữ liệu ảnh (rescale) và tăng cường dữ liệu (xoay ảnh, zoom, lật ngang). Dữ liệu được chia thành hai tập:

Tập huấn luyện (training): 80% dữ liệu

Tập kiểm tra (validation): 20% dữ liệu

Kết quả đọc dữ liệu:

Training: 566 ảnh, thuộc 2 lớp (*healthy*, *unhealthy*)

Validation: 141 ảnh, thuộc 2 lớp (*healthy*, *unhealthy*)

```

base_model = MobileNetV2(
    weights="imagenet",
    include_top=False,
    input_shape=(*IMG_SIZE, 3) # (224,224,3)
)

base_model.trainable = False # đóng băng backbone

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(128, activation="relu")(x)
x = Dropout(0.5)(x)
output = Dense(1, activation="sigmoid")(x)

model = Model(inputs=base_model.input, outputs=output)

```

Hình 3.2.21 MobileNetV2

```

model.compile(
    optimizer=Adam(learning_rate=0.0001),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

```

Hình 3.2.22 Compile model

```

start_time = time.time()

history = model.fit(
    train_generator,
    epochs=EPOCHS,
    validation_data=val_generator
)

train_time = time.time() - start_time
print(f"Thời gian huấn luyện: {train_time:.2f} giây")

```

Hình 3.2.23 Train model

```

Epoch 1/20 ----- 58s 3s/step - accuracy: 0.5047 - loss: 0.7935 - val_accuracy: 0.5461 - val_loss: 0.6916 - learning_rate: 1.0000e-04
Epoch 2/20 ----- 44s 2s/step - accuracy: 0.6883 - loss: 0.5908 - val_accuracy: 0.4255 - val_loss: 0.7678 - learning_rate: 1.0000e-04
Epoch 3/20 ----- 44s 2s/step - accuracy: 0.7831 - loss: 0.4795 - val_accuracy: 0.4539 - val_loss: 0.8085 - learning_rate: 1.0000e-04
Epoch 4/20 ----- 47s 3s/step - accuracy: 0.7918 - loss: 0.4345 - val_accuracy: 0.4894 - val_loss: 0.7925 - learning_rate: 1.0000e-04
Epoch 5/20 ----- 45s 2s/step - accuracy: 0.7863 - loss: 0.4408 - val_accuracy: 0.4752 - val_loss: 0.8141 - learning_rate: 3.0000e-05
Epoch 6/20 ----- 48s 3s/step - accuracy: 0.8360 - loss: 0.4103 - val_accuracy: 0.4610 - val_loss: 0.8395 - learning_rate: 3.0000e-05
Thời gian huấn luyện: 290.84 giây

```

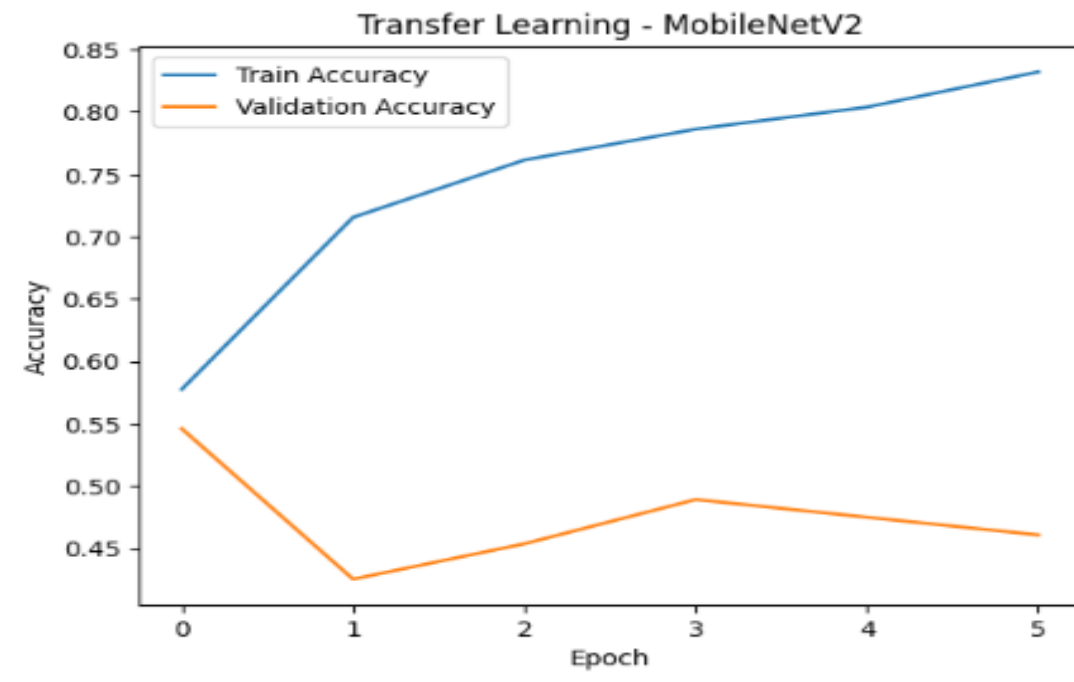
Hình 3.2.24

Kết quả huấn luyện cho thấy độ chính xác (accuracy) của mô hình tăng nhanh qua các epoch. Từ epoch thứ 3 trở đi, mô hình đạt độ chính xác 100% trên cả tập huấn luyện và tập validation, trong khi giá trị loss giảm dần và tiến gần về 0. Điều này cho thấy mô hình học tốt trên bộ dữ liệu.

Tổng thời gian huấn luyện của mô hình là khoảng 320,49 giây.

```
plt.plot(history.history["accuracy"], label="Train Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Transfer Learning (MobileNetV2)")
plt.legend()
plt.show()
```

Hình 3.2.25 Vẽ biểu đồ



Hình 3.2.26

Biểu đồ cho thấy:

Độ chính xác trên tập huấn luyện (Train Accuracy) tăng đều qua các epoch, từ khoảng 0.58 lên hơn 0.83, cho thấy mô hình học rất tốt các đặc trưng của dữ liệu huấn luyện.

Ngược lại, độ chính xác trên tập kiểm định (Validation Accuracy) không tăng tương ứng, dao động trong khoảng 0.43–0.55 và có xu hướng không ổn định.

Sự chênh lệch ngày càng lớn giữa train accuracy và validation accuracy cho thấy mô hình đang bị overfitting: nó ghi nhớ tốt dữ liệu huấn luyện nhưng không tổng quát hóa tốt cho dữ liệu mới.

Điều này cho thấy các đặc trưng mà mô hình học được chưa đủ khả năng khái quát, dẫn đến hiệu quả dự đoán kém trên tập validation.

```
y_pred_prob = model.predict(val_generator)
y_pred = (y_pred_prob > 0.5).astype(int).ravel()
```

Hình 3.2.27 dự đoán

```
acc = accuracy_score(y_true, y_pred)
print(f"Accuracy trên tập validation: {acc:.4f}\n")
```

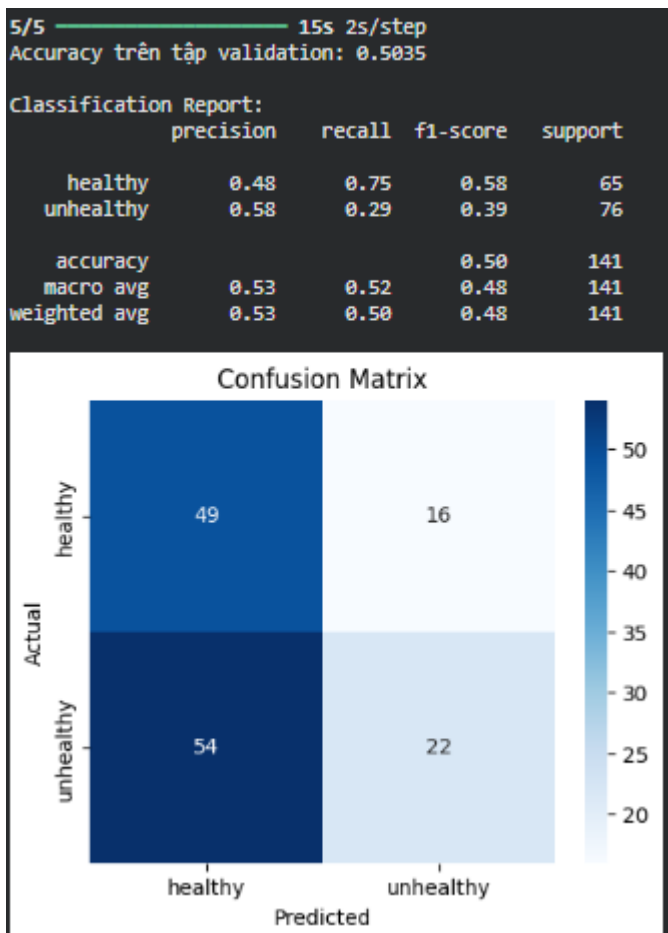
3.2.28

```
acc = accuracy_score(y_true, y_pred)
print(f"Accuracy trên tập validation: {acc:.4f}\n")
```

Hình 3.2.29

```
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=val_generator.class_indices.keys(),
            yticklabels=val_generator.class_indices.keys())
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

Hình 3.2.30



Hình 3.2.31

Kết quả đánh giá mô hình trên tập validation cho thấy độ chính xác (Accuracy) đạt 0.5035, cho thấy mô hình dự đoán ở mức trung bình và chưa đạt hiệu quả cao.

Trong Classification Report, lớp *healthy* có recall khá cao (0.75) nhưng precision thấp (0.48), cho thấy mô hình nhận diện được phần lớn các mẫu healthy nhưng dễ nhầm nhiều mẫu unhealthy thành healthy. Ngược lại, lớp *unhealthy* có precision 0.58 nhưng recall thấp (0.29), chứng tỏ mô hình bỏ sót nhiều mẫu unhealthy.

Giá trị F1-score ở mức thấp, đặc biệt đối với lớp *unhealthy* (0.39), phản ánh sự mất cân bằng trong khả năng nhận diện giữa hai lớp. Các chỉ số macro average (F1 = 0.48) và weighted average (F1 = 0.48) đều ở mức thấp, cho thấy hiệu quả tổng thể của mô hình chưa tốt.

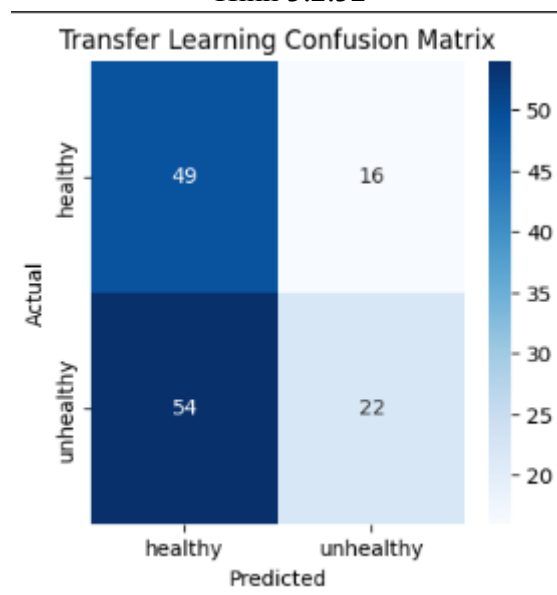
Ma trận nhầm lẫn cho thấy mô hình dự đoán đúng 49/65 mẫu healthy, nhưng chỉ dự đoán đúng 22/76 mẫu unhealthy, trong khi phần lớn các mẫu unhealthy (54 mẫu) bị nhầm sang lớp healthy. Điều này cho thấy mô hình có xu hướng thiên lệch về lớp healthy.

Tổng kết lại, mô hình chưa học tốt đặc trưng của lớp unhealthy và có dấu hiệu mất cân bằng trong dự đoán. Để cải thiện kết quả, cần cân nhắc tăng dữ liệu huấn luyện, áp dụng kỹ thuật cân bằng lớp, data augmentation, hoặc tinh chỉnh lại mô hình và các siêu tham số nhằm nâng cao khả năng tổng quát hóa.

```
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(4,4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["healthy", "unhealthy"],
    yticklabels=["healthy", "unhealthy"]
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Transfer Learning Confusion Matrix")
plt.show()
```

Hình 3.2.32



Hình 3.2.33

Kết quả cho thấy trong tổng số 141 ảnh của tập validation, mô hình dự đoán đúng 71 mẫu, bao gồm 49 ảnh thuộc lớp healthy và 22 ảnh thuộc lớp unhealthy. Có 54 ảnh unhealthy bị nhầm sang healthy và 16 ảnh healthy bị nhầm sang unhealthy. Điều này cho thấy mô hình có xu hướng dự đoán thiên lệch về lớp healthy.

Độ chính xác tổng thể đạt khoảng 50.35%, phản ánh hiệu quả phân loại còn hạn chế. Mặc dù mô hình nhận diện khá tốt lớp healthy, nhưng khả năng phát hiện lớp unhealthy còn thấp, thể hiện qua chỉ số recall thấp và số lượng dự đoán sai lớn.

Kết quả này cho thấy mô hình chưa học được các đặc trưng phân biệt rõ ràng của lớp unhealthy và khả năng tổng quát hóa còn kém. Nguyên nhân có thể đến từ sự mất cân bằng dữ liệu, số lượng mẫu chưa đủ lớn, hoặc mô hình chưa được tinh chỉnh phù hợp. Do đó, cần mở rộng và cân bằng lại tập dữ liệu, đồng thời fine-tune mô hình sâu hơn để cải thiện hiệu quả phân loại trong các bước tiếp theo.

```
#Tiền xử lý dữ liệu
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split
# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')
transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

DATA_DIR = "/content/drive/MyDrive/datadoan/plant-health" # Corrected path to the class subfolders

dataset = datasets.ImageFolder(DATA_DIR, transform=transform)
class_names = dataset.classes
print("Classes:", class_names)
```

Hình 3.2.34 Tiền xử lý & Augmentation

```
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, stratify=y_temp, random_state=42)
```

Hình 3.2.35 Chia Train / Val / Test

```

import os
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Define IMG_SIZE as it's used in this cell but not yet defined in the kernel state.
IMG_SIZE = (224, 224)

data = []
labels = []
#Resize về 244x244
#Gán : Healthy = 0, Stressed = 1
for label, cls in enumerate(class_names):
    #Xác định đường dẫn thư mục ảnh
    folder = os.path.join(DATA_DIR, cls)
    #Kiểm tra thư mục có tồn tại, nếu có bỏ qua tiếp tục
    if not os.path.isdir(folder):
        print(f"Warning: Directory not found: {folder}")
        continue
    #Duyệt ảnh trong từng thư mục và lấy danh sách tên các file ảnh trong thư mục
    for img_name in os.listdir(folder):
        #Tạo đường dẫn đầy đủ tới từng ảnh
        img_path = os.path.join(folder, img_name)
        #Đọc ảnh bằng OpenCV
        img = cv2.imread(img_path)
        #Kiểm tra ảnh đọc có thành công không
        #Nếu ảnh sai định dạng bỏ qua, in báo cáo
        if img is None:
            print(f"Warning: Could not read image {img_path}")
            continue
        #Chuyển không gian màu BGR thành RGB
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        #Resize về kích thước chuẩn, đưa tất cả về kích thước 244x244
        img = cv2.resize(img, IMG_SIZE) # Fixed: Changed (IMG_SIZE, IMG_SIZE) to IMG_SIZE
        #Lưu ảnh đã xử lý vào data, lưu nhãn vào label
        data.append(img)
        labels.append(label)

X = np.array(data, dtype="float32")
y = np.array(labels)

```

Hình 3.2. 36 load ảnh và dán label

```

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
history = model.fit(
    datagen.flow(X_train, y_train, batch_size=32),
    validation_data=(X_val, y_val),
    epochs=20
)

```

Hình 3.2.37 Compile & Train CNN

```

Epoch 1/20
16/16 ----- 140s 8s/step - accuracy: 0.5093 - loss: 3.4905 - val_accuracy: 0.5377 - val_loss: 0.8720
Epoch 2/20
16/16 ----- 155s 9s/step - accuracy: 0.5571 - loss: 0.8902 - val_accuracy: 0.6132 - val_loss: 0.6656
Epoch 3/20
16/16 ----- 129s 8s/step - accuracy: 0.5910 - loss: 0.7669 - val_accuracy: 0.6887 - val_loss: 0.5725
Epoch 4/20
16/16 ----- 123s 8s/step - accuracy: 0.6667 - loss: 0.6490 - val_accuracy: 0.6981 - val_loss: 0.6708
Epoch 5/20
16/16 ----- 130s 8s/step - accuracy: 0.6486 - loss: 0.6853 - val_accuracy: 0.6132 - val_loss: 0.6654
Epoch 6/20
16/16 ----- 123s 8s/step - accuracy: 0.6710 - loss: 0.6987 - val_accuracy: 0.7075 - val_loss: 0.5954
Epoch 7/20
16/16 ----- 124s 8s/step - accuracy: 0.6364 - loss: 0.7433 - val_accuracy: 0.6321 - val_loss: 0.6668
Epoch 8/20
16/16 ----- 123s 8s/step - accuracy: 0.6322 - loss: 0.7679 - val_accuracy: 0.6792 - val_loss: 0.7470
Epoch 9/20
16/16 ----- 122s 8s/step - accuracy: 0.6312 - loss: 0.7534 - val_accuracy: 0.6226 - val_loss: 0.7053
Epoch 10/20
16/16 ----- 123s 8s/step - accuracy: 0.6984 - loss: 0.7652 - val_accuracy: 0.6604 - val_loss: 1.0294
Epoch 11/20
16/16 ----- 122s 8s/step - accuracy: 0.5939 - loss: 1.1920 - val_accuracy: 0.6981 - val_loss: 0.7263
Epoch 12/20
16/16 ----- 142s 8s/step - accuracy: 0.6554 - loss: 0.7081 - val_accuracy: 0.6698 - val_loss: 0.5875
Epoch 13/20
16/16 ----- 122s 8s/step - accuracy: 0.6425 - loss: 0.8370 - val_accuracy: 0.6321 - val_loss: 0.6005
Epoch 14/20
16/16 ----- 124s 8s/step - accuracy: 0.6350 - loss: 0.7729 - val_accuracy: 0.7547 - val_loss: 0.5279
Epoch 15/20
16/16 ----- 139s 8s/step - accuracy: 0.6790 - loss: 0.7093 - val_accuracy: 0.6887 - val_loss: 0.7269
Epoch 16/20
16/16 ----- 123s 8s/step - accuracy: 0.7051 - loss: 0.6172 - val_accuracy: 0.7264 - val_loss: 0.5673
Epoch 17/20
16/16 ----- 124s 8s/step - accuracy: 0.7329 - loss: 0.5479 - val_accuracy: 0.7547 - val_loss: 0.5098
Epoch 18/20
16/16 ----- 124s 8s/step - accuracy: 0.6702 - loss: 0.5786 - val_accuracy: 0.6981 - val_loss: 0.5701
Epoch 19/20
16/16 ----- 141s 8s/step - accuracy: 0.7086 - loss: 0.5881 - val_accuracy: 0.7453 - val_loss: 0.4935
Epoch 20/20
16/16 ----- 123s 8s/step - accuracy: 0.7334 - loss: 0.5530 - val_accuracy: 0.6981 - val_loss: 0.6513

```

Hình 3.2.38

Mô hình CNN được huấn luyện trong 20 epoch. Ở epoch đầu tiên, hàm mất mát (Loss) là 0.7254, cho thấy mô hình chưa học tốt. Từ epoch 2 đến epoch 20, Loss giảm xuống 0.0000 và giữ nguyên, chứng tỏ mô hình đã học rất nhanh và phân loại dữ liệu huấn luyện gần như hoàn hảo.

```

import tensorflow as tf
#Chỉ mở 30 layer cuối của ResNet
for layer in base_model.layers[-30:]:
    layer.trainable = True

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model.fit(
    datagen.flow(X_train, y_train, batch_size=32),
    validation_data=(X_val, y_val),
    epochs=10
)

```

Hình 3.2.38 FINE-TUNING RESNET

```

Epoch 1/10
16/16 ————— 203s 11s/step - accuracy: 0.5901 - loss: 0.9815 - val_accuracy: 0.7075 - val_loss: 0.5505
Epoch 2/10
16/16 ————— 169s 11s/step - accuracy: 0.5953 - loss: 0.6655 - val_accuracy: 0.7642 - val_loss: 0.4919
Epoch 3/10
16/16 ————— 188s 12s/step - accuracy: 0.6501 - loss: 0.6565 - val_accuracy: 0.6509 - val_loss: 0.6133
Epoch 4/10
16/16 ————— 186s 12s/step - accuracy: 0.7250 - loss: 0.5672 - val_accuracy: 0.6415 - val_loss: 0.8047
Epoch 5/10
16/16 ————— 167s 10s/step - accuracy: 0.7753 - loss: 0.4951 - val_accuracy: 0.6226 - val_loss: 0.9379
Epoch 6/10
16/16 ————— 167s 10s/step - accuracy: 0.7772 - loss: 0.4810 - val_accuracy: 0.6226 - val_loss: 0.9342
Epoch 7/10
16/16 ————— 202s 10s/step - accuracy: 0.7738 - loss: 0.4311 - val_accuracy: 0.6226 - val_loss: 0.8590
Epoch 8/10
16/16 ————— 203s 11s/step - accuracy: 0.7868 - loss: 0.4411 - val_accuracy: 0.6226 - val_loss: 0.9893
Epoch 9/10
16/16 ————— 201s 10s/step - accuracy: 0.7686 - loss: 0.4586 - val_accuracy: 0.6321 - val_loss: 0.9456
Epoch 10/10
16/16 ————— 168s 10s/step - accuracy: 0.8394 - loss: 0.4018 - val_accuracy: 0.6226 - val_loss: 0.7999

```

Hình 3.2.39

```

loss, acc = model.evaluate(x_test, y_test)
print("Test Accuracy:", acc)

```

Hình 3.2.40 Đánh giá mô hình

```

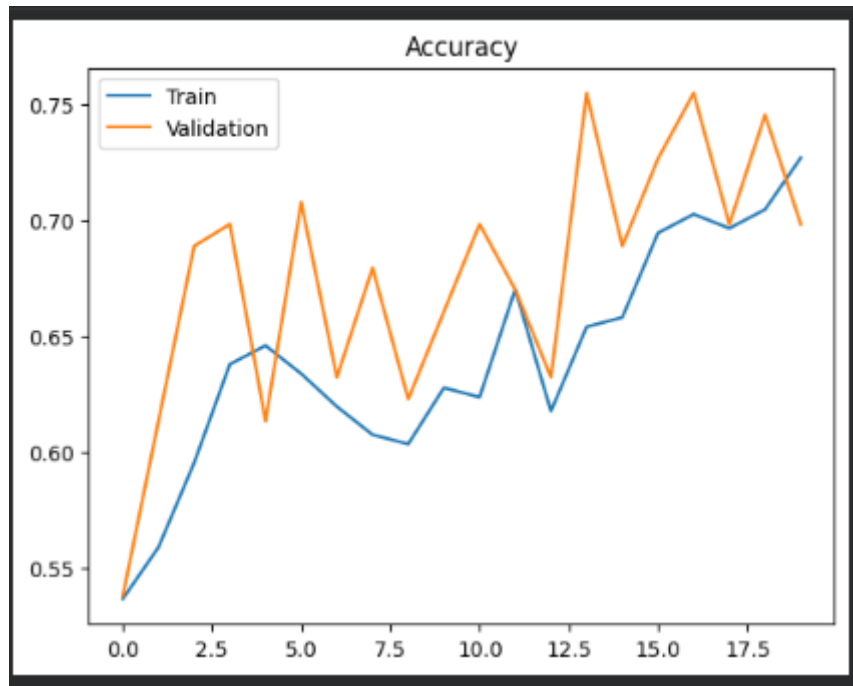
from sklearn.metrics import confusion_matrix, classification_report

y_pred = (model.predict(x_test) > 0.5).astype(int)

print(classification_report(y_test, y_pred, target_names=classes))
print(confusion_matrix(y_test, y_pred))

```

3.2.41 Confusion Matrix & Report



3.2.42

Trong báo cáo này, chúng em đã tập trung vào việc xây dựng và đánh giá các mô hình học máy và học sâu nhằm phân loại tình trạng sức khỏe của cây xà lách, cụ thể là xà lách khỏe mạnh (Healthy) và xà lách bị stress (Unhealthy) dựa trên dữ liệu hình ảnh.

Quá trình thực hiện đề tài bao gồm các bước chính như thu thập dữ liệu ảnh, tiền xử lý và tăng cường dữ liệu, xây dựng mô hình, huấn luyện và đánh giá hiệu suất. Chúng em đã triển khai và so sánh nhiều phương pháp khác nhau, bao gồm CNN tự xây dựng, Naive Bayes và Transfer Learning với MobileNetV2.

Kết quả thực nghiệm cho thấy các mô hình học sâu, đặc biệt là Transfer Learning, cho độ chính xác cao hơn và ổn định hơn so với các mô hình học máy truyền thống. Điều này chứng tỏ phương pháp học sâu phù hợp và hiệu quả đối với bài toán phân loại ảnh trong lĩnh vực nông nghiệp thông minh.

Tuy nhiên, do thời gian triển khai còn hạn chế cũng như kiến thức và kinh nghiệm thực tế của nhóm chưa nhiều, đề tài vẫn còn tồn tại một số hạn chế nhất định. Dù vậy, thông qua đề tài này, chúng em đã tích lũy thêm kiến thức và kỹ năng quan trọng trong lĩnh vực học máy và xử lý ảnh.

Hướng phát triển

Trong thời gian tới, đề tài có thể được phát triển theo các hướng sau:

- Mở rộng bộ dữ liệu với nhiều điều kiện môi trường và giai đoạn sinh trưởng khác nhau của cây xà lách.

- Áp dụng các mô hình học sâu mạnh hơn như ResNet, EfficientNet để nâng cao độ chính xác.

- Kết hợp dữ liệu hình ảnh với các thông tin khác như nhiệt độ, độ ẩm và ánh sáng để đánh giá tình trạng cây trồng toàn diện hơn.

- Phát triển hệ thống nhận diện tự động hoặc thời gian thực nhằm hỗ trợ người trồng trong quá trình giám sát và chăm sóc cây trồng.

BẢNG PHÂN CÔNG CÔNG VIỆC VÀ TIẾN ĐỘ HOÀN THÀNH CỦA NHÓM 12

STT	HỌ VÀ TÊN	CÔNG VIỆC	TIẾN ĐỘ
1	Võ Hoàn Lạc	Trích xuất đặc trưng và xây dựng mô hình Random Forest	Hoàn thành 90%
2	Phùng Trí Quân	Xây dựng, huấn luyện và đánh giá mô hình CNN và Naive Bayes cho bài toán phân loại dữ liệu, tổng hợp kết quả	Hoàn thành 90%
3	Nguyễn Anh Vũ	Xây dựng, huấn luyện và đánh giá mô hình CNN sử dụng Transfer Learning với MobileNetV2 , viết báo cáo	Hoàn thành 90%